

RACoon⁺: A System for LLM-based Table Understanding with a Knowledge Graph

Lindsey Linxi Wei
University of Washington
linxiwei@cs.washington.edu

Guorui Xiao
University of Washington
grxiao@cs.washington.edu

Moe Kayali
University of Washington
kayali@cs.washington.edu

Dan Suciu
University of Washington
suciu@cs.washington.edu

Magdalena Balazinska
University of Washington
magda@cs.washington.edu

ABSTRACT

Recent advances in large language models (LLMs) have opened new opportunities for addressing tasks that require semantic understanding of data, such as table understanding tasks (e.g., column-type annotation). Existing LLM-based methods, however, rely mainly on the model’s internal parametric knowledge, ignoring external context that could help disambiguate cell content. Knowledge graphs (KGs) are a potential source of structured, factual data about real-world entities and their relationships, which can help with semantic data understanding. In this paper, we design, implement, and evaluate **RACoon⁺**, a prototype system that jointly leverages the parametric knowledge in LLMs and the non-parametric facts stored in an external KG to perform table understanding tasks. Experiments with our prototype show that **RACoon⁺** significantly improves over state-of-the-art LLM-based baselines.

PVLDB Reference Format:

Lindsey Linxi Wei, Guorui Xiao, Moe Kayali, Dan Suciu, and Magdalena Balazinska. RACoon⁺: A System for LLM-based Table Understanding with a Knowledge Graph. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/lindseylinxiwei/RACoon>.

1 INTRODUCTION

In recent years, researchers and data scientists have demonstrated that Web data contains a large number of relational tables [2, 16, 17]. These tables can serve for downstream tasks such as data quality control [26] and data discovery [5]. However, important information such as semantic column types and column-pair relations, which are often used in commercial systems [46], is often missing from these tables [33]. The recovery of this semantic information is called table understanding: the task of identifying correspondences between table elements and a predefined vocabulary [12]. Two important subtasks in table understanding are column type annotation (CTA) and relation extraction (RE) [30, 33]. CTA assigns

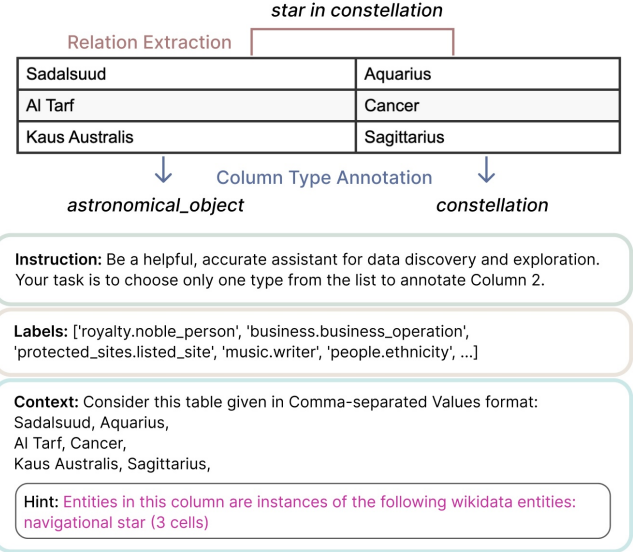


Figure 1: The upper part is an illustration of CTA and RE. The lower part is an example prompt for CTA, consisting of the instruction, labels, and context (with hint) sections.

one or more semantic types to each column in a table (e.g., the first column in Figure 1 has the semantic type *astronomical_object*), while RE annotates column pairs with one or more relations (e.g., the first column and the second column in Figure 1 have the semantic relation *star in constellation*). There has been increasing interest recently in developing methods that can automatically perform table understanding tasks. Early approaches [22, 42] used probabilistic models with hand-crafted features to jointly perform CTA and RE. Subsequent methods fine-tuned pre-trained language models (PLMs) on task-specific datasets to capture the contextual information of a table and improve semantic type and relation annotation performance [30, 33, 43]. The recent development of *Large Language Models* (LLMs) [28, 41, 44] has led to models that are pretrained over a large text corpus to generate the next token in an auto-regressive way based on a user’s input in natural language, or *prompt*, without requiring any task-specific fine-tuning. Early work that used LLMs for table understanding [7, 18, 39, 40, 52] has explored prompting-based LLM methods, where prompts are constructed with task instructions, table context, and a label set for

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

the LLM to generate predictions. Figure 1 shows an example (we discuss the hint in the prompt below).

Although these methods show great promise, at their core, they assume that the parametric knowledge encoded in LLMs is sufficient and thus apply them without any external knowledge; this paradigm has been shown to produce factual inaccuracies [34], generate outdated responses [36], and struggle with domain-specific or specialized queries [19]. For example, without access to structured knowledge about astronomy, an LLM might fail to identify the obscure star names in the first column in Figure 1, and label it with "visual_artist" instead. To alleviate such problems, Retrieval-Augmented Generation (RAG) [4, 20, 29] includes additional up-to-date and domain-specific context in the prompt. In this paradigm, an external knowledge source is required: Given a user’s query, a retriever ranks source documents by similarity, selects the most relevant ones, processes them, and augments the prompt to the LLM with the extracted information. Throughout this process, the LLM’s internal parameters remain unchanged.

Among the many external knowledge sources, Knowledge Graphs (KGs) can provide up-to-date factual knowledge in the form of entities (nodes) and relations (edges) [23], where each edge is a factual triplet of the form $\langle \text{subject}, \text{relation}, \text{object} \rangle$, making KGs well-structured data sources. Continuing our example, for each cell in the first column in Figure 1, we can retrieve a triplet, such as $\langle \text{Al Tarf}, \text{instance of}, \text{navigational star} \rangle$, which provides information on the column type and can be added as a hint to the LLM.

Inspired by advances in LLMs and the benefits of RAG, we design, implement, and evaluate **RACoon**⁺, a RAG-based system for *LLM-based table understanding* that leverages both the power of LLMs and factual knowledge from a KG to annotate semantic information in tables. Given a relational table, a table understanding task, an LLM, and a KG, **RACoon**⁺ retrieves relevant information from the KG and incorporates it into the LLM prompt to enhance its performance on the task. Note that **RACoon**⁺ extends our preliminary prototype that originally appeared in a four-page workshop paper [49], in which we only briefly discussed how RAG can improve LLM-based solutions for a single table understanding task without addressing the two main challenges described below.

Building **RACoon**⁺ poses two key challenges:

(1) KG Retrieval. Accurately linking entity mentions in table cells to their reference entities in a KG is a challenging yet crucial step in retrieval, since the meaning of a cell value can vary significantly depending on the table context. For instance, the entity mention "Japan" may refer to the country when appearing in a column with other country names. In contrast, in a different table about football tournament champions, where the column contains names of national teams, "Japan" refers to the Japanese national football team rather than the country itself. As such, directly applying an off-the-shelf entity linking method over cell values cannot reliably distinguish between these different entities. To address this challenge, we build a tabular-structure-aware KG-Linker that incorporates not only the cell text but also additional table context. We explore multiple strategies for using table context and demonstrate their effectiveness in improving both entity linking precision and end-to-end performance.

(2) Post-retrieval refinement. Even if the retriever returns factual triplets from a KG, this information is not directly ready for use.

Different table understanding tasks benefit from different kinds of knowledge, and the retrieved set may also contain semantically misaligned or misleading facts due to imperfect entity linking. To address both issues, **RACoon**⁺ carefully selects retrieved KG information according to the requirements of the specific table understanding task and employs an LLM-powered pruning component to filter out semantically misaligned information, ensuring that only the most useful knowledge is incorporated into the prompt.

In summary, this paper makes the following contributions:

(1) We introduce the problem of augmenting LLM prompts with external knowledge retrieved from a KG to improve performance on table understanding tasks. We then propose and implement our end-to-end prototype system **RACoon**⁺ to solve this problem on column type annotation and relation extraction (Section 3).

(2) We develop an effective retrieval mechanism that takes advantage of different types of table context to retrieve relevant facts from the KG at multiple levels of granularity, tailored to each table understanding task (Section 4.2).

(3) We design dedicated post-retrieval modules that enable **RACoon**⁺ to dynamically select the most relevant KG information for each table understanding task, allowing the LLM to achieve higher accuracy on both CTA and RE (Section 4.3).

(4) We evaluate **RACoon**⁺ on popular table understanding benchmarks and demonstrate that it consistently outperforms baselines by up to 27.91 Micro F1 (Section 5).

2 BACKGROUND

We introduce the two table understanding tasks supported by **RACoon**⁺, along with background on LLM-based table understanding and RAG with a KG. Table 1 summarizes the notation.

Table understanding tasks aim to discover the *semantic* attributes of data contained in Web tables [22, 30, 33, 43], which often lack metadata including captions and table headers [38]. In this paper, we assume that only table instances are available (i.e., the cell values in T). A *table* T is a collection of columns $C_1, \dots, C_n$, where each *column* is an array of values. Thus, a traditional record in T is $(C_1[i], C_2[i], \dots, C_n[i])$. In addition, for each table understanding task, we are given a set of *labels* L . We focus on two common tasks: column type annotation (CTA) and relation extraction (RE). **RACoon**⁺’s framework is extensible to other tasks.

Column Type Annotation [7, 9, 18, 30, 33] focuses on annotating columns with semantic types:

Definition 2.1. Given a relational table T with columns $C_1, \dots, C_n$ and a label set L of semantic types, column type annotation (CTA) aims to assign each column C_i one or more types $l \in L$ such that each value in C_i is an instance of l .

Relation Extraction [30, 33] focuses on assigning semantic relation labels to column pairs:

Definition 2.2. Given a table T with columns $C_1, C_2, \dots, C_n$. Following earlier work [30], we define C_1 as the subject column, and $C_2, \dots, C_n$ as the object columns. Given a label set L of semantic relations, relation extraction (RE) aims to annotate each subject-object column pair with a relation $r \in L$, such that r holds between all entity pairs in the column pair.

Symbol	Description
$T = (C_1, \dots, C_n)$	A relational table with n columns C_i , each with an array of cell values, forming records.
I	Table understanding task instructions in natural language.
L	Label set of the table understanding task. In CTA, L contains <i>semantic</i> type labels. In RE, L contains <i>semantic</i> column pair relation labels.
G	A knowledge graph that consists of a set of triplets t .
$t = \langle s, p, o \rangle$	A KG triplet with subject s , predicate p , and object o .
h	Hint derived from external knowledge retrieved from a KG in natural language.
AU	Annotation unit. The target column for CTA or the target subject-object column pair for RE.

Table 1: Notation summary.

Large Language Models (LLMs) LLMs are transformer-based models pre-trained on vast text corpora [6, 28]. After *instruction fine-tuning* [14], they show strong performance on tasks for which they were not explicitly trained. Recent approaches [7, 18] have started to harness this capability for table understanding tasks. As shown in Figure 1, a typical table understanding task prompt consists of three components: (1) **Instructions**, a natural-language description of the task; (2) **Labels**, the set of candidate labels the model must choose for annotation; and (3) **Context**, the contextual information provided to the model for annotation. Prior methods assume that the relational table T alone is sufficient as the **Context**. In contrast, as we show in this paper, providing additional context inside the prompt with external knowledge from a KG leads to significant gains. Therefore, in this work, we define **Context** to consist of both the relational table T and an additional hint h . This hint provides additional semantic context about the table column(s), helping the LLM better understand their content, types, or relationships.

Retrieval-Augmented Generation (RAG) LLMs have been shown to fail on knowledge-intensive tasks that are domain-specific [19] or require recent factual information not included inside the training data [36]. RAG [20, 29] was proposed to alleviate these issues by incorporating a reliable external knowledge source. Typically, during the generation stage of an LLM, RAG works by first measuring similarity between the input prompt and a set of external documents that form a source of knowledge [37] and retrieving the top- k closest documents. Then, the retrieved content is cleaned, filtered and re-ranked [11] to form the final additional context to append to the original prompt. Recent work [53] shows that the dense, structured facts in a knowledge graph can provide robust knowledge for Question-Answer(QA) task. Thus in **RACoon**⁺, we similarly use a KG as the external knowledge source to retrieve factual triplets that are relevant to the table columns. By connecting structured table data with structured KG data through RAG, **RACoon**⁺ leverages both the structural signals of tables and the relational knowledge in the KG to improve LLM’s performance on table understanding tasks.

Knowledge Graph (KG) KGs, such as Wikidata [47] and DBpedia [24], store large collections of real-world entities and facts.

They come in triplet format: $\langle \text{subject}, \text{predicate}, \text{object} \rangle$. The subject and object are nodes—either entities or literal values (e.g., $\langle \text{Seattle}, \text{population}, 744955 \rangle$), and the predicate is a directed edge representing the relationship between them [47]. Every entity in the KG has a unique identifier. In Wikidata, entities are identified by QIDs, such as Q5083 for "Seattle." There may also be metadata such as descriptions that add context in addition to the label. KGs have been shown to be particularly effective in improving performance of various tasks when used as knowledge sources for RAG [8, 13, 15, 25, 43].

3 RACoon⁺’S APPROACH

We now formalize the *RAG-enhanced table-understanding* problem and present **RACoon**⁺’s approach. All symbols used below are summarized in Table 1.

At a high level, **RACoon**⁺ retrieves additional context from an external knowledge graph (KG), refines it, and supplements the provided table context to improve end-to-end task performance. Formally, we are given a relational table T with columns $(C_1, \dots, C_n)$, containing a bag of tuples. Each cell in each tuple may be a string, numerical value, or datetime value. Although some tables include captions or headers, such metadata is often missing [38]. Thus, **RACoon**⁺ assumes this information is not given. Let G be a knowledge graph represented as triplets $\langle s, p, o \rangle$, and let I be a plain-language description of the table understanding task (e.g., a description of CTA or RE). Before querying the LLM, **RACoon**⁺ retrieves a subgraph $G' \subset G$, prunes irrelevant information from G' to get $G'' \subseteq G'$, and converts it into a textual hint h conditioned on both T and G'' based on the specific table understanding task. Finally, **RACoon**⁺ constructs the prompt (T, I, h, L) for CTA and RE. As we show in the paper, this constructed prompt can then be used for a variety of downstream models, yielding consistent improvements in end-to-end table-understanding performance. Nonetheless, **RACoon**⁺’s approach is not limited to CTA and RE, but can be naturally extended to other table understanding tasks. However, due to space constraint, we leave it for future work.

4 RACoon⁺’S SYSTEM

In this section, we describe **RACoon**⁺’s overall workflow, as shown in Figure 2, and the details of its components. Inspired by the advanced RAG paradigm from the NLP community [31], **RACoon**⁺ employs three stages to retrieve external knowledge from the KG: Pre-Retrieval, Retrieval, and Post-Retrieval. **RACoon**⁺ then leverages the retrieved information to generate a hint, which it adds to the LLM prompt.

4.1 Pre-Retrieval

RAG is only effective on columns whose cells correspond to real-world entities that can be linked to a KG, which we call *entity columns*. Columns that do not meet this criterion (e.g., numerical values, date/time values, or long descriptive text) are *non-entity columns*, and including them in retrieval introduces noise and misleading hints. For example, a numeric column [2012, 2357, 1407, 1362] should be labeled as "identifier," but a KG-Linker might incorrectly map "1407" to the Wikidata entity "United Nations Security Council Resolution 1407," which can hurt performance since LLMs

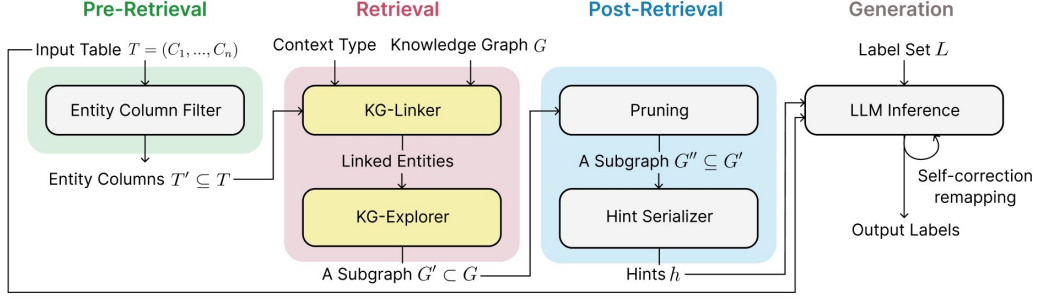


Figure 2: RACoon⁺ consists of four stages: Pre-Retrieval, which extracts entity columns from the input table; Retrieval, where the KG-Linker links each cell in T' to its referent entity in the KG and the KG-Explorer retrieves a subgraph from the linked entities within each annotation unit; Post-Retrieval, which prunes irrelevant information from the subgraph and serializes it into natural language hints; and Generation, where an LLM takes the hints, input table, and label set to answer a table understanding task, followed by a self-correction remapping step if the predicted label falls outside the given label set.

Q184602 Entity Label: Iran men's national football team Entity Description: men's national association football team representing Iran	Hybrid Context Linking <table><tr><td>← Preceded by Iran</td><td>West Asian Championship Champions 2002</td><td>Succeeded by Iran</td></tr><tr><td>...</td><td>...</td><td>...</td></tr><tr><td>← Preceded by Korea Republic DPR</td><td>Asian Games Champions 1982</td><td>Succeeded by Korea Republic</td></tr></table>	← Preceded by Iran	West Asian Championship Champions 2002	Succeeded by Iran	...	...	...	← Preceded by Korea Republic DPR	Asian Games Champions 1982	Succeeded by Korea Republic
← Preceded by Iran	West Asian Championship Champions 2002	Succeeded by Iran								
...	...	...								
← Preceded by Korea Republic DPR	Asian Games Champions 1982	Succeeded by Korea Republic								
Q543842 Entity Label: South Korea men's national football team Entity Description: men's national association football team representing South Korea	Cell Context Linking <table><tr><td>Q884</td><td>Entity Label: South Korea</td><td>Entity Description: country in East Asia</td></tr></table>	Q884	Entity Label: South Korea	Entity Description: country in East Asia						
Q884	Entity Label: South Korea	Entity Description: country in East Asia								

Figure 3: Example table and linking results for the first column (other columns omitted).

are sensitive to irrelevant context [10]. To address this, **RACoon⁺** introduces an Entity Column Filter in the pre-retrieval stage. Simple heuristics (e.g., Pandas type inference or token length thresholds) can handle obvious cases but fail on hybrid columns containing both numerical and textual descriptors (e.g., [Runtime: 86 mins, Runtime: 76 mins]). We instead use an LLM-based filter that serializes each column and prompts the LLM to decide whether cells in the column represent real-world entities. As shown in Figure 2, the filter selects a subset of entity columns $T' \subseteq T$ for the KG-Linker in the Retrieval stage.

4.2 Retrieval

The retrieval stage collects relevant information from the KG for each *Annotation Unit (AU)* in the set of entity columns T' . An AU is either a single column for CTA or a column pair for RE. For example, in Figure 1, the first column is an AU for CTA, while the first and second columns together form an AU for RE. The retrieval process proceeds in two steps: (1) Linking each cell in T' to its corresponding KG entity using the KG-Linker. (2) For each AU in T' , traversing the KG from the linked entities with the KG-Explorer to construct a subgraph containing task-relevant information.

4.2.1 KG-Linker. The inputs to the KG-Linker include the entity columns T' , an external knowledge graph G , and a context type, which is defined later in this section. The output is a list of KG identifiers (QIDs in Wikidata), each representing the entity linked to a

corresponding cell in T' , as illustrated in Figure 3. State-of-the-art entity linking (EL) methods typically rely on Transformer-based models [27, 50], which use bi-directional attention to encode text. This allows the model to process text in both directions simultaneously and consider the entire context surrounding each word [32]. As a result, the contextualized embeddings generated by these models are sensitive to surrounding text, so the same token can have different representations depending on its context. A key challenge in directly applying an off-the-shelf Transformer-based EL model to each cell value from a query table is that it fails to consider the rich semantic information encoded in the table, leading to suboptimal performance. For example, if we aim to link the cell "Proceeded by Japan" in the first column of the table in Figure 3 and only provide the cell text to the EL model, we will get the Wikidata entity Q17: "Japan: island country in East Asia." Such linking result does not capture the semantics of the cell within the query table, which is about football tournament champions. To leverage the semantic information of an input query table, we study the benefits of using four different table-specific context types for constructing the input to the EL model: cell, table, column, and hybrid.

Table Context. Using the whole table as context allows the EL model to build richer contextual representations of table cell tokens than with cell context, improving linking accuracy (Table 5), but its precision remains relatively low. Precision is important in end-to-end performance, as it is preferable to leave cells unlinked rather than introduce incorrect entities [51]. Large tables often produce noisy serialized contexts, which can overwhelm the EL model with irrelevant information and degrade linking precision. Given the inherent structure of relational tables, where cells within the same row or column are semantically related, we turn to more targeted context types.

Column Context. Cells within a column tend to be semantically homogeneous. Thus, a table column forms a focused and relevant context for each cell, which helps the EL model disambiguate entities more accurately, improving the linking precision. For example, the cell "Pulse" alone may be linked to Q191733 ("arterial palpation of the heartbeat"), but with the column context ["Pulse", "The Ring", "Shall We Dance", "Hachi: A Dog's Tale"], the linker correctly maps

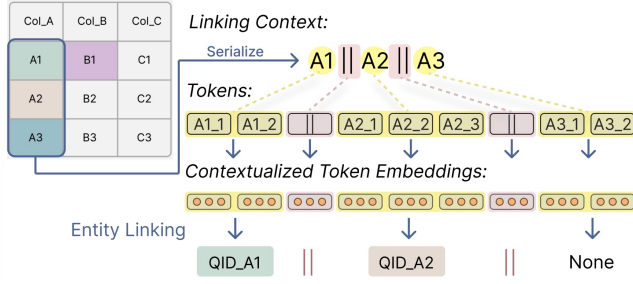


Figure 4: KG-Linker with column context. The first column Col_A , with cells (A1, A2, A3), is serialized into the string "A1||A2||A3", where "||" is a cell delimiter. We first identify delimiter tokens and record their positions to determine token spans for each cell. Since each token maps to a contextualized embedding, these spans are used to separate the embedding sequence into per-cell segments. Entity linking is then performed on each cell's embedding segment, resulting in a linked knowledge graph entity if one is found (e.g., A1 is linked to QID_{A1}), or None if no entity is found (e.g., A3).

it to Q744911 ("film directed by Jim Sonzero"). However, in some cases, the column context is too semantically uniform to reflect the contextual meaning of the cells in the broader table context. For instance, in Figure 3, the first column lists only country names, so the linker maps each cell to a country entity, ignoring the broader table semantics about football tournaments. On the other hand, we also consider the row context, which captures cross-column information but cannot take advantage of column homogeneity from column context.

Hybrid Context. To balance these complementary strengths and limitations, we propose a hybrid context that combines information from both the column and the row. By jointly considering the column homogeneity and the inter-column relations, the hybrid context best reflects the table semantics. As discussed in Section 5.2.2, **RACoon**⁺'s KG-Linker with hybrid context achieves the best end-to-end results.

Once the context type is specified, the KG-Linker uses it for all cells in T' , and all cells within the context scope are handled simultaneously. For example, with column context, it performs EL on each column of T' at once. One challenge in applying off-the-shelf EL models is that they are designed for free text, not structured tables. Once tabular data are flattened into plain text, these models lose cell boundaries and may incorrectly link mentions across different cells. To address this, we introduce a cell boundary tracking technique to preserve cell boundaries during entity linking.

The input to the EL model is serialized by concatenating the table cells within the scope defined by the chosen context type, using the delimiter "||" to separate individual cells. For example, the first column of the table in Figure 3 will be serialized into the text: "Precededby Iran || ... || Precededby Korea Republic Korea DPR". After tokenization of this serialized linking context, the positions of the delimiter tokens are recorded to determine the token spans for each cell. In Figure 4, the cell A1 has token spans [A1_1, A1_2]. Since each token maps to a contextualized embedding, these token

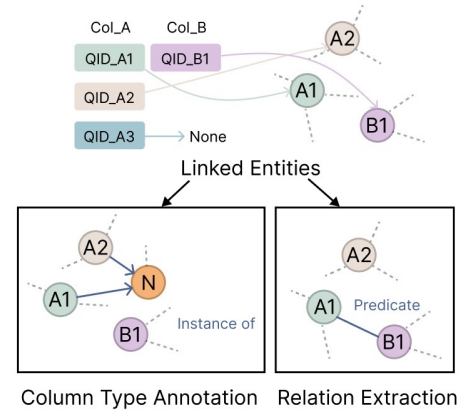


Figure 5: Illustration of KG-Explorer with different table understanding tasks. The three table cells A1, A2 from Col_A and B1 from Col_B are linked to three nodes in the KG. In CTA, the KG-Explorer uses the *instance of* predicate to hop on the KG and get the *Entity Type* N. A1 and A2 are instances of N. Note that N is an existing node in the KG. In RE, the KG-Explorer obtains the predicate between a cell pair (A1, B1).

spans are used to separate the embedding sequence into per-cell segments. Then, entity linking is performed over each cell's embedding segment as illustrated in Figure 4. Finally, we obtain a mapping from each cell in T' to a corresponding entity in G represented by its KG identifier (QID for Wikidata), or to None if no such entity exists. In our prototype implementation, we use ReFinED¹ [27] as the EL model in the KG-Linker.

As discussed in Section 2, each linked entity in the KG encodes information through metadata (labels, descriptions) and triplets that specify its relations with other entities. To leverage this information for table understanding tasks, **RACoon**⁺ introduces the KG-Explorer.

4.2.2 KG-Explorer. The KG-Explorer processes each AU (Section 4.2) and maps it to a subgraph G' in the knowledge graph G . We define G' as the set of nodes corresponding to the linked entities in the AU, along with their metadata and directly connected neighbors through predicates. By traversing the KG from these linked entities, the KG-Explorer retrieves information at multiple levels of granularity. Specifically, **RACoon**⁺ makes use of four types of information: *Entity Label*, *Entity Description*, *Entity Type*, and *Entity Relation*.

Entity Label. The simplest information we can retrieve from the KG is the entity's label, which provides a canonical representation of ambiguous or incomplete cell mentions. For example, the entity mention '15 Sge' is unclear on its own, but its linked *Entity Label* is '15_Sagittae' in Wikidata, making it easier for the LLM to interpret.

Entity Description. Beyond labels, each entity in the KG also comes with a description that we refer to as *Entity Description*, which is a short phrase designed to disambiguate items with similar labels. For example, '15_Sagittae' has the description "star in the constellation Sagitta". In cases where the *Entity Label* is ambiguous, the *Entity Description* provides additional context to help the LLM better understand the semantics of the entity.

¹<https://github.com/amazon-science/ReFinED>

Entity Type. In addition to labels and descriptions, entities are associated with semantic types. The KG-Explorer follows the *instance of* predicate to retrieve each entity’s type, which directly informs the column type. For example, Q170566 “Japan men’s national football team” has the triplet $\langle \text{Japan men’s national football team, instance of, National association football team} \rangle$, where the object is the *Entity Type*. If an entity has multiple types, we keep only the one most frequent among other cells in the same column.

Entity Relation. Finally, the KG-Explorer can also examine each pair of cells from the same row across two columns and retrieves the predicate connecting their corresponding entities in the KG. Since RE aims to annotate the relation between a subject-object column pair, retrieving predicates based on individual cell pairs provides direct hints about the semantic relationship. For example, from the triplet $\langle \text{Rustom Padilla, place of birth, Philippines} \rangle$, it retrieves the predicate “place of birth.”

As illustrated in Figure 5, the output of the KG-Explorer is a set of mapping from each AU in T' to a subgraph $G' \subset G$. In our experiments shown in Table 5, we explore various combinations of these types of information to assess their impact on LLM’s performance.

4.3 Post-Retrieval

When the KG-Linker generates incorrect linkings between cells and KG entities, these linkings can cause the KG-Explorer to retrieve misleading information that conflicts with the table’s semantics. For example, if the cell “Precededby Korea Republic Korea DPR” is incorrectly linked to Q884 “South Korea”, the retrieved *Entity Type* “country” would contradict the intended column semantics of football teams and misguide the LLM, leading to sub-optimal task performance. To address this issue, **RACoon**⁺ introduces a Post-Retrieval stage to assess the consistency of each retrieved subgraph G' with the column data and the table context in the Pruning component, and then integrate it as natural language hints for the LLM with the Hint Serializer.

Pruning. The goal of this component is to remove retrieved content that is clearly incorrect, irrelevant, or redundant with respect to the table. We initially explored using inter-column relations for pruning in the CTA task, based on the intuition that shared relations between columns indicate more reliable linking results and lead to relevant retrieved information. Specifically, for each cell with a linked entity, we check whether the entity has a predicate in the KG that connects it to entities linked from other cells in the same row. If such a relation exists, we retain the linked entity for that cell. However, this relation-based filtering did not yield meaningful gains in our preliminary experiments, because inter-column relations are often sparse in real-world tables, causing useful information to be pruned along with noisy links.

Given these limitations, we turned to an LLM-based approach. The LLM is given the entire table T and the retrieved subgraph G' and instructed to discard information that does not align with the actual column values or the broader table context (e.g., incorrect facts, unrelated entities, or duplicates). The result is a cleaned subgraph $G'' \subseteq G'$. A detailed prompt template is available in our GitHub repository².

Hint Serializer. This component converts the pruned subgraph

Table 2: Datasets used in the evaluation

Dataset	# of tables	# of instances	# of labels
TURL-127-CTA [30]	4764	13025	127
TURL-66-RE [30]	1467	2072	66
SOTAB V2-CTA [38]	609	1851	82
SOTAB V2-RE [38]	565	2340	108

G'' into a natural language hint h using predefined templates. For example, in CTA, if G'' contains the *Entity Type* national association football team linked to 5 cells, the hint is: “Entities in this column are instances of: national association football team (5 cells).” Full templates for all tasks are available in our GitHub repository.

4.4 Generation

The final stage of **RACoon**⁺’s workflow is Generation. The inputs are the input table T , the label set L , and the hints h we obtained from the Post-Retrieval stage. The output is the LLM’s prediction for each AU (a column for CTA or a subject-object column pair for RE). **RACoon**⁺ constructs a prompt by combining the inputs in the order of **Instructions**, **Labels**, and **Context**, where the **Context** includes both the original table content and the hints h . As illustrated in Figure 1, this augmented prompt is then passed to the LLM to produce predictions.

Self-correction remapping. If the initial output of the LLM is not in the given label set L , **RACoon**⁺ follows existing work [7] to perform label remapping. Specifically, **RACoon**⁺ calls the LLM again with the original chat history, appending a new message that instructs the LLM to select the closest valid label from L . For example, the initial LLM output “human settlement”, which is not in L , will be mapped to the label “location” on the second attempt.

5 EVALUATION

Datasets. We evaluate **RACoon**⁺ on two widely used table understanding benchmarks, TURL [30] and SOTAB V2 [38], using their full test sets. Dataset statistics are reported in Table 2. In TURL, we observe substantial semantic overlap in the label sets (e.g., “time.event” vs. “time.recurring_event”). To address this, we follow [7] to reduce semantic overlap: For TURL-CTA, remapping 225 original labels to 127 labels; for TURL-RE, remapping 121 original labels to 66 labels. Details of the remapping are available in our GitHub repository.

Baselines. We compare **RACoon**⁺ against zero-shot LLM inference on CTA and RE without any external knowledge. Note that all the methods evaluated in this section, including ours and the baselines, are in a zero-shot setting. However, **RACoon**⁺ can also be extended to a few-shot in-context learning setting.

We implement two baselines: NoHint performs generation without hints. SelfHint first prompts an LLM with the input table to predict a coarse type (for CTA) or relation (for RE) for each AU (Section 4.2). The coarse predictions are then used as hints in the Generation stage. Comparison against SelfHint evaluates how much additional value the external knowledge from a KG provides beyond what the LLM can infer on its own.

Since the TURL-127-CTA and TURL-66-RE benchmarks include ground-truth entity linking information for each table cell to the

²<https://github.com/lindseylinxiwei/RACoon>

Table 3: Main results for CTA and RE

Benchmark	Method (w/ GPT-3.5-turbo)	Micro F1
TURL-127-CTA	NoHint	50.25
	SelfHint	59.58
	RACoon ⁺ + Ground-truth-linker	79.87
	RACoon ⁺ + ReFinED _{hybrid}	78.16
TURL-66-RE	NoHint	51.89
	SelfHint	52.30
	RACoon ⁺ + Ground-truth-linker	66.99
	RACoon ⁺ + ReFinED _{hybrid}	65.63
SOTAB V2-CTA	NoHint	62.76
	SelfHint	60.45
	RACoon ⁺ + ReFinED _{column}	66.30
SOTAB V2-RE	NoHint	66.49
	SelfHint	56.69
	RACoon ⁺ + ReFinED _{column}	66.82

Wikidata KG, we also create a **RACoon**⁺ variant that leverages this information, denoted as Ground-truth-linker. This third baseline approximates the upper bound of **RACoon**⁺’s performance assuming a perfect KG-Linker.

Evaluation Metrics. Following existing work on table understanding [7, 21, 30, 33], we use the Micro F1 score as our evaluation metric since it is less affected by variations in the number of examples per label and the performance of individual labels. Furthermore, each test instance in TURL-CTA and TURL-RE datasets can have one or more overlapping ground truth labels, and to make the tasks more suitable for LLM, we evaluate *all LLM-based methods* as follows: If its prediction falls within the set of ground truth labels, we consider it to be correct; otherwise, we consider it to be incorrect.

Implementation. We implement **RACoon**⁺ using Python 3.12. We use GPT-4o-mini in the Entity Column Filter and the Pruning components. To compare fairly with existing methods [7, 18, 39, 40], we use GPT-3.5-turbo in the Generation stage for both CTA and RE, but we also experiment with other newer GPT models and an open-source Llama model. We conduct our experiments on a server with a single A40 GPU with 8 cores and 128GB RAM.

5.1 Main Results

The main results are shown in Table 3 and we highlight in bold the best-performing methods (except for the Ground-truth-linker, which is only there to provide an upper bound on **RACoon**⁺’s performance assuming a perfect linker). In this end-to-end experiment, we use the best-performing configuration of **RACoon**⁺.

CTA. On TURL-127-CTA, with ReFinED_{hybrid} as the KG-Linker, **RACoon**⁺ outperforms NoHint by 28.07 Micro F1 and SelfHint by 18.74 Micro F1. Furthermore, it is only 1.71 Micro F1 lower than the **RACoon**⁺ using Ground-truth-linker, showing the effectiveness of our table-aware ReFinED_{hybrid}. On SOTAB V2-CTA, since ground truth cell linking to the knowledge graph is not available, we evaluate **RACoon**⁺ directly using the best-performing ReFinED_{column} as the KG-Linker. **RACoon**⁺ outperforms the NoHint baseline by 3.54 and the SelfHint baseline by 5.85.

Table 4: Comparison of KG-Linkers on end-to-end performance (Micro F1), without Pruning

KG-Linker	TURL-127-CTA	TURL-66-RE
Ground-truth	79.87	66.99
MediaWiki API	62.49	56.32
ReFinED _{cell}	71.80	54.83
ReFinED _{table}	67.19	59.10
ReFinED _{column}	72.96	60.66
ReFinED _{hybrid}	73.72	62.47

Table 5: Entity Linking performance of KG-Linkers on TURL

KG-Linker	MediaWiki	ReFinED			
	Cell	Cell	Table	Column	Hybrid
Accuracy	2.28	47.99	55.43	54.00	57.81
Precision	45.01	79.75	80.31	80.49	82.89
Recall	5.07	60.17	69.02	67.09	68.33

RE. On TURL-66-RE, with ReFinED_{hybrid}, **RACoon**⁺ achieves 65.63 Micro F1, significantly outperforming NoHint by 13.74 and SelfHint by 13.33. Furthermore it is only 1.36 F1 lower than the **RACoon**⁺ using Ground-truth-linker. In SOTAB V2-RE, **RACoon**⁺ with ReFinED_{column} achieves 66.82 Micro F1, outperforming the SelfHint baseline by 10.13 while slightly outperforming the NoHint baseline.

The key difference between the TURL and SOTAB benchmarks is that TURL contains entities from Wikipedia and the KG that we use in the experiments, Wikidata, is based on Wikipedia. In contrast, SOTAB V2 tables are derived from a different structured knowledge source Schema.org [38]. Interestingly, **RACoon**⁺ still provide performance gain even when the KG and datasets do not match, especially on the CTA task, but the performance boost from **RACoon**⁺ is much higher when the KG is aligned with the dataset, as in TURL.

Overall, these results highlight three key findings: (1) **RACoon**⁺ consistently improves end-to-end performance compared to both NoHint and SelfHint baselines; (2) **RACoon**⁺ shows robust improvements across a variety of table understanding tasks; and (3) with its table-aware linker, **RACoon**⁺ can leverage external knowledge and achieves performance close to that of a ground-truth linker.

5.2 Detailed Results

This section presents a detailed analysis of the effects of various design choices on each component in **RACoon**⁺.

5.2.1 Entity Column Filter. We further analyze the impact of different Entity Column Filters described in Section 4.1 over SOTAB V2-CTA, where each table instance contains a mixture of entity and non-entity columns. Without any filter, **RACoon**⁺ reaches 62.97 Micro F1. The rule-based filter gives a modest gain, reaching 64.38 Micro F1, while the LLM-based filter achieves the best result with 66.30 Micro F1. This suggests that (1) applying a filter consistently improves performance over the no-filter setting and (2) using LLMs for filtering entity columns is more reliable than simple heuristics.

Table 6: Comparison of model choices (Micro F1)

Benchmark	Method	GPT-3.5-turbo	Llama-3.1-8B	GPT-4.1-nano	GPT-4o-mini
TURL-127-CTA	NoHint	50.25	47.79	47.25	71.22
	RACoon ⁺ + Ground-truth-linker	79.87	74.94	82.45	85.83
	RACoon ⁺ + ReFinED _{hybrid}	78.16	69.51	76.68	80.53
TURL-66-RE	NoHint	51.89	52.83	37.36	62.11
	RACoon ⁺ + Ground-truth-linker	66.99	66.20	51.93	69.15
	RACoon ⁺ + ReFinED _{hybrid}	65.63	63.67	48.39	67.60

5.2.2 *KG-Linker*. We study how different KG-Linkers with various context types affect the end-to-end performance (Table 4). We implement one additional baseline named MediaWiki API ³, where **RACoon**⁺ queries the Wikidata KG by sending the cell content through its search API and use the top-1 query result as the retrieved entity. Table 5 shows the accuracy, precision, and recall of different KG-Linkers on the columns where the NoHint baseline failed. Experiments show that a table-aware linker with higher precision consistently leads to higher Micro F1. For example, on TURL-127-CTA, **RACoon**⁺ equipped with ReFinED_{hybrid} has the highest linking precision, and it outperforms **RACoon**⁺ with MediaWiki API by 11.23 Micro F1. We observe the same trend on RE. Since ReFinED_{hybrid} considers both row-wise and column-wise context, it outperforms **RACoon**⁺ with other linker by a large margin.

Table 7: Analysis of different types of information retrieved by the KG-Explorer in Micro F1

	TURL-127-CTA	TURL-66-RE
<i>Entity Label</i>	72.56	57.96
<i>Entity Label + Description</i>	70.08	57.21
<i>Entity Type</i>	78.16	62.06
<i>Entity Relation</i>	69.35	65.63

We evaluate the effect of different combinations of retrieved information (entity label, entity description, entity type, and entity relation) from the KG. Table 7 shows the results. Interestingly, because the CTA task requires annotating a *column* with a semantic type label, using retrieved *Entity Type*, which aggregates semantic type information from the column’s cells, gives the highest Micro F1. On the other hand, the RE task aims to annotate semantic relations between pairs of columns, so *Entity Relation*, which captures relationships between pairs of cells, achieves the best Micro F1. This demonstrates that retrieved information is most effective when it is aligned with the task objective.

5.2.3 *Choice of models*. We analyze the effectiveness of **RACoon**⁺ across different models in Table 6. In general, more advanced models lead to stronger end-to-end performance across tasks (e.g., GPT-4o-mini with NoHint outperforms GPT-3.5-turbo with NoHint by 20.97 Micro F1). Importantly, **RACoon**⁺ with ReFinED_{hybrid} also benefits from these improvements and consistently outperforms the baselines across different model choices. This demonstrates

the robustness of **RACoon**⁺ to model selection and that it remains effective even as models advance.

6 RELATED WORK

Table Understanding. Table understanding [9, 16, 18, 30, 33, 52] has long been a popular research area. Recently, researchers have shown promising results using large language models (LLMs) for table understanding [7, 18, 21, 39, 40]. One series of works [39, 40] apply LLMs to column type annotation and relation extraction directly. Building on this, CHORUS [18] proposes a general framework that leverages chat histories for column type annotation and related tasks, while ArcheType [7] offers a dedicated pipeline specifically for column type annotation. These methods assume that table content plus the parametric knowledge of the LLM is sufficient. In contrast, **RACoon**⁺ retrieves additional external knowledge during LLM generation. Another line of research fine-tunes LLMs on task-specific augmented data [21], or retrieves in-context examples to enhance LLM performance at inference time [1], yet still omits the use of external knowledge at generation time. Thus, **RACoon**⁺ is orthogonal to and complementary with all of these approaches.

Retrieval Augmented Generation (RAG). Despite the promising performance of LLMs over unseen tasks, they often struggle with generating factually inaccurate answers [34], answering domain-specific queries [19], and dealing with outdated knowledge [36]. To address these problems, RAG was proposed, aiming to augment LLMs with additional related context relative to a user’s query [4, 20, 29, 35, 45]. More recently, GraphRAG [3] proposes a graph-based RAG approach to address global sensemaking queries over large text corpora, which are not well supported by conventional vector-based RAG. It constructs a knowledge graph from key entities and relations in the corpus and leverages community-level summaries to generate a global answer. Recent works [8, 48] proposed to retrieve from a knowledge graph to improve LLM’s performance over table-related tasks. KGLink [48] proposed to utilize an external KG with PLM *during training* for CTA. KG-RAG4SM [8] proposed to trace an external KG to identify the most relevant subgraph to the input query to an LLM specifically for the schema matching task. On the other hand, **RACoon**⁺ focuses on both CTA and RE, and it proposes context used for linking specifically for tables and retrieves different types of knowledge from a KG during generation with LLM. Thus **RACoon**⁺ is considered orthogonal to these works.

7 CONCLUSION

We introduce **RACoon**⁺, an LLM-based system for retrieval-augmented generation for a variety of table understanding tasks. **RACoon**⁺ first

³<https://www.wikidata.org/>

identifies entity columns within a query table, selects table-specific linking context, retrieves additional hints guided by the selected context, aggregates the hints, and finally prompts the LLM to produce the result. Experiment results show that **RACoon**⁺ consistently outperforms existing methods and baselines across different tasks and models.

REFERENCES

- [1] Amir Babamahmoudi, Davood Rafiei, and Mario A Nascimento. [n.d.]. Improving Column Type Annotation Using Large Language Models. *Proceedings of the VLDB Endowment*. ISSN 2150 ([n. d.]), 8097.
- [2] Chandra Sekhar Bhagavatula, Thanapon Noraset, and Doug Downey. 2015. TabEL: Entity Linking in Web Tables. In *The Semantic Web - ISWC 2015 - 14th International Semantic Web Conference, Bethlehem, PA, USA, October 11-15, 2015, Proceedings, Part I (Lecture Notes in Computer Science)*, Marcelo Arenas et al. (Ed.), Vol. 9366. Springer, 425–441. https://doi.org/10.1007/978-3-319-25007-6_25
- [3] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitanansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130* (2024).
- [4] Akari Asai et al. 2023. Retrieval-based Language Models and Applications. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics: Tutorial Abstracts, ACL 2023, Toronto, Canada, July 9-14, 2023*, Yun-Nung Vivian Chen et al. (Ed.), Association for Computational Linguistics, 41–46. <https://doi.org/10.18653/V1/2023.ACL-TUTORIALS.6>
- [5] Adriane Chapman et al. 2020. Dataset search: a survey. *VLDB J.* 29, 1 (2020), 251–272. <https://doi.org/10.1007/S00778-019-00564-X>
- [6] Abhimanyu Dubey et al. 2024. The Llama 3 Herd of Models. *CoRR* abs/2407.21783 (2024). <https://doi.org/10.48550/ARXIV.2407.21783> arXiv:2407.21783
- [7] Benjamin Feuer et al. 2024. ArcheType: A Novel Framework for Open-Source Column Type Annotation using Large Language Models. *Proc. VLDB Endow.* 17, 9 (2024), 2279–2292. <https://doi.org/10.14778/3665844.3665857>
- [8] Chuangtao Ma et al. 2025. Knowledge Graph-based Retrieval-Augmented Generation for Schema Matching. *CoRR* abs/2501.08686 (2025). <https://doi.org/10.48550/ARXIV.2501.08686> arXiv:2501.08686
- [9] Dan Zhang et al. 2020. Sato: Contextual Semantic Type Detection in Tables. *Proc. VLDB Endow.* 13, 11 (2020), 1835–1848. <http://www.vldb.org/pvldb/vol13/p1835-zhang.pdf>
- [10] Freda Shi et al. 2023. Large Language Models Can Be Easily Distracted by Irrelevant Context. In *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA (Proceedings of Machine Learning Research)*, Andreas Krause et al. (Ed.), Vol. 202. PMLR, 31210–31227. <https://proceedings.mlr.press/v202/shi23a.html>
- [11] Jialin Dong et al. 2024. Don't Forget to Connect! Improving RAG with Graph-based Reranking. *CoRR* abs/2405.18414 (2024). <https://doi.org/10.48550/ARXIV.2405.18414> arXiv:2405.18414
- [12] Jixiong Liu et al. 2023. From tabular data to knowledge graphs: A survey of semantic table interpretation tasks and methods. *J. Web Semant.* 76 (2023), 100761. <https://doi.org/10.1016/J.WEBSEM.2022.100761>
- [13] Jiahuo Sun et al. 2024. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=nnVO1PvbTv>
- [14] Jason Wei et al. 2022. Finetuned Language Models are Zero-Shot Learners. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net. <https://openreview.net/forum?id=gEzrGCozdqR>
- [15] Mohammad Dehghan et al. 2024. EWEK-QA : Enhanced Web and Efficient Knowledge Graph Retrieval for Citation-based Question Answering Systems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, Lun-Wei Ku et al. (Ed.), Association for Computational Linguistics, 14169–14187. <https://doi.org/10.18653/V1/2024.ACL-LONG.764>
- [16] Michael J. Cafarella et al. 2008. Uncovering the Relational Web. In *11th International Workshop on the Web and Databases, WebDB 2008, Vancouver, BC, Canada, June 13, 2008*. <http://webdb2008.com.polimi.it/images/stories/WebDB2008/paper30.pdf>
- [17] Michael J. Cafarella et al. 2008. WebTables: exploring the power of tables on the web. *Proc. VLDB Endow.* 1, 1 (2008), 538–549. <https://doi.org/10.14778/1453856.1453916>
- [18] Moe Kayali et al. 2024. CHORUS: Foundation Models for Unified Data Discovery and Exploration. *Proc. VLDB Endow.* 17, 8 (2024), 2104–2114. <https://doi.org/10.14778/3659437.3659461>
- [19] Nikhil Kandpal et al. 2023. Large Language Models Struggle to Learn Long-Tail Knowledge. In *International Conference on Machine Learning, ICLR 2023, 23-29 July 2023, Honolulu, Hawaii, USA (Proceedings of Machine Learning Research)*, Andreas Krause et al. (Ed.), Vol. 202. PMLR, 15696–15707. <https://proceedings.mlr.press/v202/kandpal23a.html>
- [20] Patrick Lewis et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, Hugo Larochelle et al. (Ed.), <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
- [21] Peng Li et al. 2024. Table-GPT: Table Fine-tuned GPT for Diverse Table Tasks. *Proc. ACM Manag. Data* 2, 3 (2024), 176. <https://doi.org/10.1145/3654979>
- [22] Petros Venetis et al. 2011. Recovering Semantics of Tables on the Web. *Proc. VLDB Endow.* 4, 9 (2011), 528–538. <https://doi.org/10.14778/2002938.2002939>
- [23] Quan Wang et al. 2017. Knowledge Graph Embedding: A Survey of Approaches and Applications. *IEEE Trans. Knowl. Data Eng.* 29, 12 (2017), 2724–2743. <https://doi.org/10.1109/TKDE.2017.2754499>
- [24] Sören Auer et al. 2007. DBpedia: A Nucleus for a Web of Open Data. In *The Semantic Web, 6th International Semantic Web Conference, 2nd Asian Semantic Web Conference, ISWC 2007 + ASWC 2007, Busan, Korea, November 11-15, 2007 (Lecture Notes in Computer Science)*, Karl Aberer et al. (Ed.), Vol. 4825. Springer, 722–735. https://doi.org/10.1007/978-3-540-76298-0_52
- [25] Shirui Pan et al. 2024. Unifying Large Language Models and Knowledge Graphs: A Roadmap. *IEEE Trans. Knowl. Data Eng.* 36, 7 (2024), 3580–3599. <https://doi.org/10.1109/TKDE.2024.3352100>
- [26] Sebastian Schelter et al. 2018. Automating Large-Scale Data Quality Verification. *Proc. VLDB Endow.* 11, 12 (2018), 1781–1794. <https://doi.org/10.14778/3229863.3229867>
- [27] Tom Ayoola et al. 2022. ReFinED: An Efficient Zero-shot-capable Approach to End-to-End Entity Linking. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Industry Track, NAACL 2022, Hybrid: Seattle, Washington, USA + Online, July 10-15, 2022*, Anastassia Loukina et al. (Ed.), Association for Computational Linguistics, 209–220. <https://doi.org/10.18653/V1/2022.NAACL-INDUSTRY.24>
- [28] Tom B. Brown et al. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, Hugo Larochelle et al. (Ed.), <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>
- [29] Weijia Shi et al. 2024. REPLUG: Retrieval-Augmented Black-Box Language Models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), NAACL 2024, Mexico City, Mexico, June 16-21, 2024*, Kevin Duh et al. (Ed.), Association for Computational Linguistics, 8371–8384. <https://doi.org/10.18653/V1/2024.NAACL-LONG.463>
- [30] Xiang Deng et al. 2022. TURL: Table Understanding through Representation Learning. *SIGMOD Rec.* 51, 1 (2022), 33–40. <https://doi.org/10.1145/3542700.3542709>
- [31] Yunfan Gao et al. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. *CoRR* abs/2312.10997 (2023). <https://doi.org/10.48550/ARXIV.2312.10997>
- [32] Yinhan Liu et al. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *CoRR* abs/1907.11692 (2019). <http://arxiv.org/abs/1907.11692>
- [33] Yoshihiko Suhara et al. 2022. Annotating Columns with Pre-trained Language Models. In *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*, Zachary G. Ives, Angela Bonifati, and Amr El Abbadi (Eds.), ACM, 1493–1503. <https://doi.org/10.1145/3514221.3517906>
- [34] Ziwei Ji et al. 2023. Survey of Hallucination in Natural Language Generation. *ACM Comput. Surv.* 55, 12 (2023), 248:1–248:38. <https://doi.org/10.1145/3571730>
- [35] Kelvin et al. Guu. 2020. Retrieval augmented language model pre-training. In *International conference on machine learning*. PMLR, 3929–3938.
- [36] Hangfeng He, Hongming Zhang, and Dan Roth. 2023. Rethinking with Retrieval: Faithful Large Language Model Inference. *CoRR* abs/2301.00303 (2023). <https://doi.org/10.48550/ARXIV.2301.00303> arXiv:2301.00303
- [37] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Trans. Big Data* 7, 3 (2021), 535–547. <https://doi.org/10.1109/TBDATA.2019.2921572>
- [38] Christian Bizer Keti Korini, Ralph Peeters. 2023. WDC Schema.org Table Annotation Benchmark Version 2 (SOTAB V2). <https://webdatacommons.org/structureddata/sotab/v2/>
- [39] Ketu Korini and Christian Bizer. 2023. Column Type Annotation using ChatGPT. In *Joint Proceedings of Workshops at the 49th International Conference on Very Large Data Bases (VLDB 2023), Vancouver, Canada, August 28 - September 1, 2023 (CEUR Workshop Proceedings)*, Rajesh Bordawekar et al. (Ed.), Vol. 3462. CEUR-WS.org. <https://ceur-ws.org/Vol-3462/TADA1.pdf>
- [40] Ketu Korini and Christian Bizer. 2024. Column Property Annotation Using Large Language Models. In *The Semantic Web: ESWC 2024 Satellite Events -*

- Hersonissos, Crete, Greece, May 26-30, 2024, *Proceedings, Part I (Lecture Notes in Computer Science)*, Albert Meroño-Peñuela et al. (Ed.), Vol. 15344. Springer, 61–70. https://doi.org/10.1007/978-3-031-78952-6_6
- [41] Jieh-Sheng Lee. 2024. InstructPatentGPT: Training patent language models to follow instructions with human feedback. *CoRR abs/2406.16897* (2024). <https://doi.org/10.48550/ARXIV.2406.16897> arXiv:2406.16897
 - [42] Girija Limaye, Sunita Sarawagi, and Soumen Chakrabarti. 2010. Annotating and Searching Web Tables Using Entities, Types and Relationships. *Proc. VLDB Endow.* 3, 1 (2010), 1338–1347. <https://doi.org/10.14778/1920841.1921005>
 - [43] Zhengjie Miao and Jin Wang. 2023. Watchog: A Light-weight Contrastive Learning based Framework for Column Annotation. *Proc. ACM Manag. Data* 1, 4 (2023), 272:1–272:24. <https://doi.org/10.1145/3626766>
 - [44] Alec et al. Radford. 2018. Improving language understanding by generative pre-training. (2018).
 - [45] Ori et al. Ram. 2023. In-context retrieval-augmented language models. *Transactions of the Association for Computational Linguistics* 11 (2023), 1316–1331.
 - [46] Tableau Software. [n.d.]. Tableau: Visual Analytics Platform. <https://www.tableau.com/>.
 - [47] Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: a free collaborative knowledgebase. *Commun. ACM* 57, 10 (2014), 78–85. <https://doi.org/10.1145/2629489>
 - [48] Yubo Wang, Hao Xin, and Lei Chen. 2024. KGLink: A Column Type Annotation Method that Combines Knowledge Graph and Pre-Trained Language Model. In *40th IEEE International Conference on Data Engineering, ICDE 2024, Utrecht, The Netherlands, May 13-16, 2024*. IEEE, 1023–1035. <https://doi.org/10.1109/ICDE60146.2024.00083>
 - [49] Lindsey Linxi Wei, Guorui Xiao, and Magdalena Balazinska. [n.d.]. RACoon: An LLM-based Framework for Retrieval-Augmented Column Type Annotation with a Knowledge Graph. In *NeurIPS 2024 Third Table Representation Learning Workshop*.
 - [50] Ledell et al. Wu. 2019. Scalable zero-shot entity linking with dense entity retrieval. *arXiv preprint arXiv:1911.03814* (2019).
 - [51] Siye et al. Wu. 2024. How easily do irrelevant inputs skew the responses of large language models? *arXiv preprint arXiv:2404.03302* (2024).
 - [52] Guorui et al. Xiao. 2025. Cents: A Flexible and Cost-Effective Framework for LLM-Based Table Understanding. *Proc. VLDB Endow.* 18, 11 (Sept. 2025), 4574–4587. <https://doi.org/10.14778/3749646.3749714>
 - [53] Xiangrong et al. Zhu. 2025. Knowledge Graph-Guided Retrieval Augmented Generation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Luis et al. Chiruzzo (Ed.). Association for Computational Linguistics, Albuquerque, New Mexico, 8912–8924. <https://doi.org/10.18653/v1/2025.naacl-long.449>